DYNAMIC_PORT_BINDING_AND_SERVICE_DI

Dynamic Port Binding and Service Discovery Architecture

Version: 1.0
Date: 2025-11-07
Status: Design Phase

Table of Contents

Executive Summary

This document outlines the architecture for a dynamic port binding and service discovery system for HelixCode's distributed services. The system enables services to:

- **Dynamically allocate ports** when default ports are unavailable
- **Broadcast configuration** to dependent services
- **Discover services** via multiple mechanisms (default port, broadcast, registry)
- **Handle errors gracefully** with fallback strategies
- **Work across environments** (local, Docker, Kubernetes, distributed)

Key Benefits

- **Zero-configuration** service startup in most cases
- **Resilient** to port conflicts and network changes
- **Testable** with comprehensive test coverage
- **Production-ready** with monitoring and observability

Problem Statement

Current Limitations

1. **Fixed Port Configuration**: Services use hard-coded ports (e.g., PostgreSQL:5432, Redis:6379)
2. **Port Conflicts**: When default ports are occupied, services fail to start
3. **Manual Configuration**: Developers must manually update configurations when ports change
4. **Testing Challenges**: Running multiple test suites in parallel causes port conflicts
5. **Docker Limitations**: Port mapping requires manual orchestration

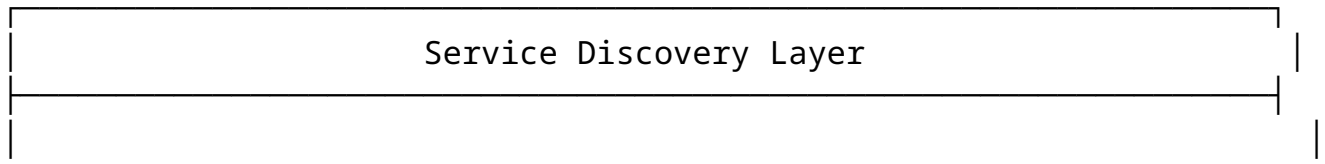
Requirements

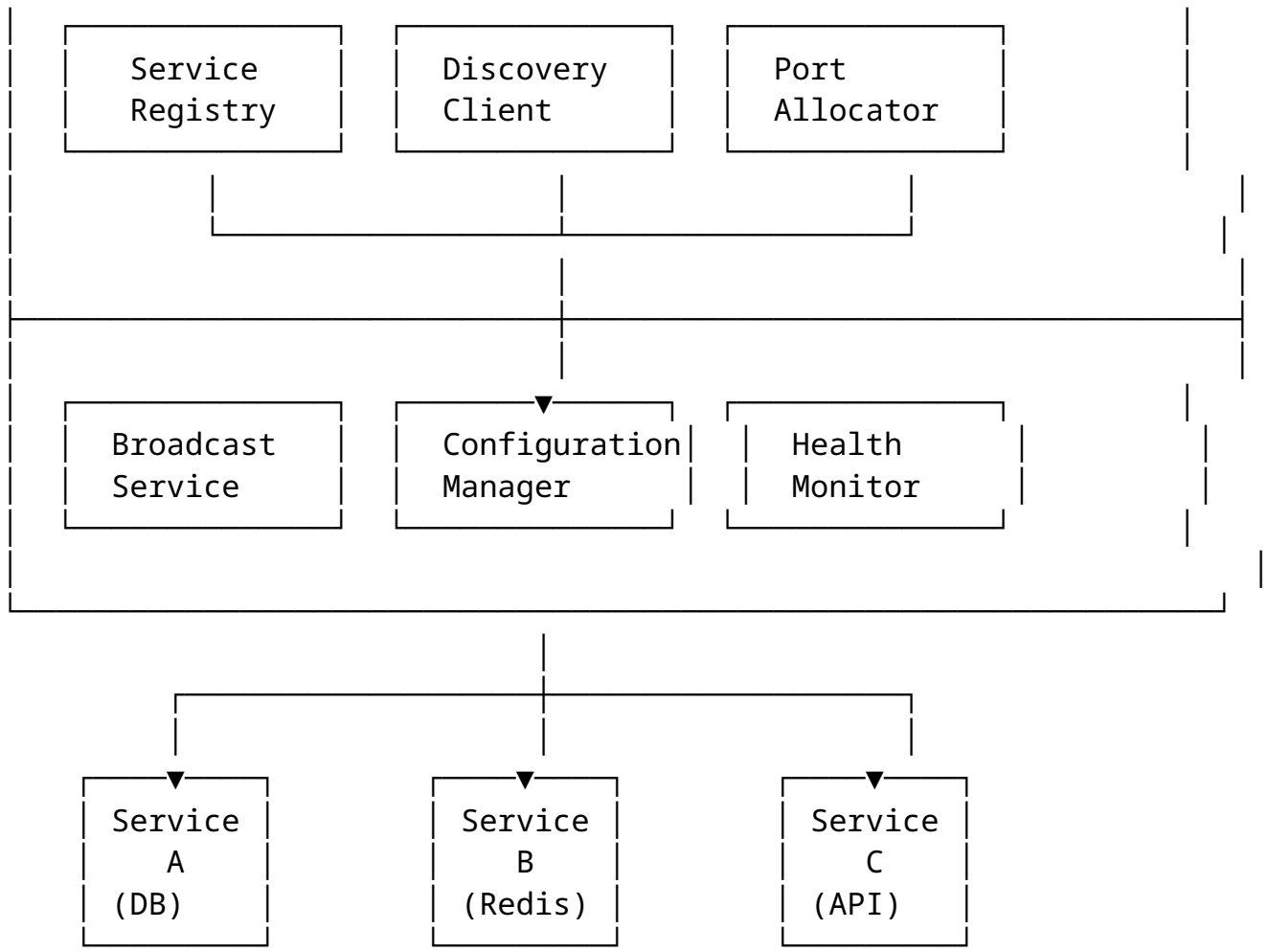
The solution must:

1. Automatically bind to first available port if default is taken
2. Broadcast service configuration to dependent services
3. Support service discovery via multiple mechanisms
4. Handle error states and provide fallback options
5. Be fully tested (unit, integration, E2E, automation)
6. Be comprehensively documented
7. Work in all deployment scenarios (local, Docker, Kubernetes)
8. Support both synchronous and asynchronous discovery
9. Provide monitoring and observability

Architecture Overview

High-Level Design





Components Interaction

1. **Service A** starts and requests port allocation
2. **Port Allocator** finds first available port (fallback if default is taken)
3. **Service A** registers with **Service Registry**
4. **Broadcast Service** announces Service A's configuration
5. **Service B** discovers Service A via:
    - Default port attempt
    - Registry lookup
    - Broadcast listener
6. **Configuration Manager** updates dependent services
7. **Health Monitor** tracks service availability

Core Components

1. Port Allocator

Location: `internal/discovery/port_allocator.go`

Responsibilities: - Allocate ports with fallback mechanism - Check port availability - Manage port ranges - Handle port reservations - Support named port pools (e.g., “database”, “cache”, “api”)

Key Functions:

```
type PortAllocator interface {
    AllocatePort(serviceName string, preferredPort int) (int,
        error)
    AllocatePortInRange(serviceName string, startPort, endPort
        int) (int, error)
    ReleasePort(port int) error
    IsPortAvailable(port int) bool
    GetPortForService(serviceName string) (int, bool)
}
```

2. Service Registry

Location: `internal/discovery/registry.go`

Responsibilities: - Store service metadata (name, host, port, version) - Support service registration and deregistration - Health check integration - TTL-based expiration - Multi-protocol support (HTTP, gRPC, custom)

Data Model:

```
type ServiceEndpoint struct {
    ID           string
    Name         string
    Host         string
    Port         int
    Protocol     string
    Version      string
    Metadata     map[string]string
    HealthURL    string
    RegisteredAt time.Time
    LastHeartbeat time.Time
    TTL          time.Duration
}
```

3. Discovery Client

Location: `internal/discovery/client.go`

Responsibilities: - Discover services by name - Support multiple discovery strategies - Implement retry logic with exponential backoff - Cache discovered endpoints - Handle service failover

Key Functions:

```go
type DiscoveryClient interface {
    Discover(serviceName string, opts ...DiscoverOption)
        (*ServiceEndpoint, error)
    DiscoverAll(serviceName string) ([]*ServiceEndpoint, error)
    Watch(serviceName string) (<-chan ServiceEvent, error)
    Close() error
}
```

4. Broadcast Service

Location: `internal/discovery/broadcast.go`

Responsibilities: - Broadcast service announcements (UDP multicast) - Listen for service announcements - Support multiple broadcast channels - Handle network segmentation

Protocol:

```json
{
  "type": "service_announcement",
  "service": {
    "name": "postgres-primary",
    "host": "localhost",
    "port": 5434,
    "protocol": "tcp",
    "metadata": {
      "database": "helixcode",
      "version": "16.0"
    }
  },
  "timestamp": "2025-11-07T12:00:00Z",
  "ttl": 60
}
```

5. Configuration Manager

Location: `internal/discovery/config_manager.go`

Responsibilities: - Manage service configurations dynamically - Update configurations when services change - Support hot-reload of configurations - Validate configuration changes

6. Health Monitor

Location: `internal/discovery/health.go`

Responsibilities: - Monitor service health - Update registry with health status - Trigger failover on health check failures - Support custom health check strategies

---

Port Allocation Mechanism

Port Selection Strategy

1. **Try Default Port**: Attempt to bind to the preferred/default port
2. **Try Port Range**: If default is unavailable, scan port range
3. **Random Selection**: For large ranges, use random selection with validation
4. **Ephemeral Ports**: Fall back to OS-assigned ephemeral ports if configured

Port Ranges by Service Type

Service Type	Default Port	Fallback Range	Pool Name
PostgreSQL	5432	5433-5442	database
Redis	6379	6380-6389	cache
HTTP API	8080	8081-8099	api
gRPC	9090	9091-9109	grpc
Metrics	9090	9100-9199	metrics
WebSocket	8000	8001-8020	websocket

Port Allocation Algorithm

```
func (pa *PortAllocator) AllocatePort(serviceName string,
        preferredPort int) (int, error) {
    // 1. Check if preferred port is available
    if pa.IsPortAvailable(preferredPort) {
        pa.reservePort(preferredPort, serviceName)
        return preferredPort, nil
    }

    // 2. Check if service already has a port assigned
```

```
    if existingPort, exists :=
        pa.GetPortForService(serviceName); exists {
        return existingPort, nil
    }

    // 3. Get fallback range for service type
    serviceType := pa.getServiceType(serviceName)
    startPort, endPort := pa.getPortRange(serviceType)

    // 4. Scan range for available port
    for port := startPort; port <= endPort; port++ {
        if pa.IsPortAvailable(port) {
            pa.reservePort(port, serviceName)
            return port, nil
        }
    }

    // 5. If no port found in range, try ephemeral
    if pa.config.AllowEphemeral {
        return pa.allocateEphemeralPort(serviceName)
    }

    return 0, ErrNoPortsAvailable
}
```

Port Availability Check

```
func (pa *PortAllocator) IsPortAvailable(port int) bool {
    // Try to bind to the port
    listener, err := net.Listen("tcp", fmt.Sprintf(":%d", port))
    if err != nil {
        return false
    }
    listener.Close()
    return true
}
```

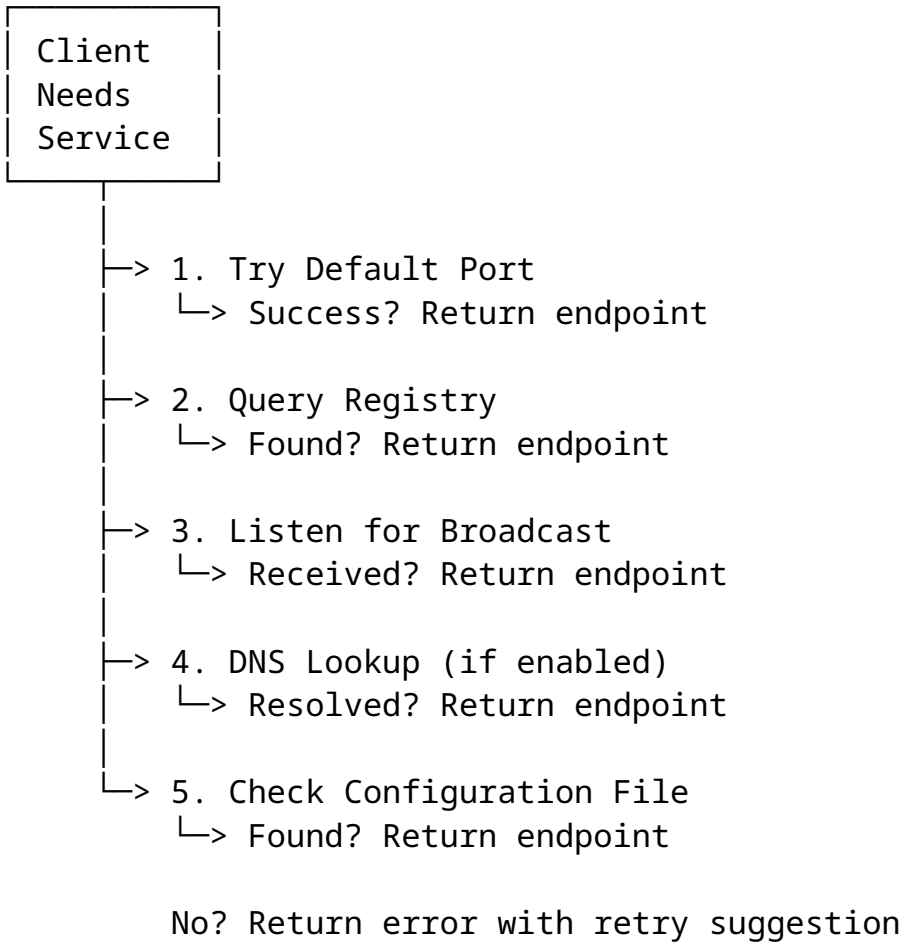
Service Discovery Protocol

Discovery Strategies

The system supports multiple discovery strategies, attempted in order:

1. **Default Port** (fastest, no overhead)
2. **Registry Lookup** (centralized, reliable)
3. **Broadcast Discovery** (peer-to-peer, works across networks)
4. **DNS Discovery** (for Kubernetes/Docker Swarm)
5. **Configuration File** (fallback, manual)

Discovery Workflow



Example: Discovering PostgreSQL

```
// Client code
client, _ := discovery.NewClient(discovery.Config{
    RegistryURL: "http://localhost:7000",
    BroadcastEnabled: true,
    Strategies: []string{"default", "registry", "broadcast"},
```

```
})

// Discover PostgreSQL service
endpoint, err := client.Discover("postgres-primary",
    discovery.WithTimeout(5 * time.Second),
    discovery.WithRetries(3),
)

if err != nil {
    log.Fatal("Failed to discover PostgreSQL:", err)
}

// Use the discovered endpoint
dsn := fmt.Sprintf("postgres://user:pass@%s:%d/db",
    endpoint.Host, endpoint.Port)
```

Configuration Broadcasting

Broadcast Mechanism

Services broadcast their configuration using **UDP multicast** to a well-known address:

- **Multicast Address**: `239.255.0.1`
- **Port**: `7001`
- **Protocol**: JSON over UDP

Broadcast Message Format

```
{
  "version": "1.0",
  "type": "service_announcement",
  "action": "register",
  "service": {
    "id": "postgres-primary-abc123",
    "name": "postgres-primary",
    "host": "192.168.1.100",
    "port": 5434,
    "protocol": "tcp",
    "version": "16.0",
    "metadata": {
      "database": "helixcode",
      "sslmode": "disable",
```

```
      "max_connections": "100"
    },
    "health_url": "http://192.168.1.100:5434/health"
  },
  "timestamp": "2025-11-07T12:00:00Z",
  "ttl": 60,
  "signature": "sha256_hash_for_verification"
}
```

Broadcast Actions

1. **register**: Service comes online
2. **deregister**: Service going offline gracefully
3. **update**: Configuration change
4. **heartbeat**: Periodic keep-alive

Broadcast Listener

```
type BroadcastListener struct {
    conn     *net.UDPConn
    handlers map[string]AnnouncementHandler
    stop     chan bool
}

func (bl *BroadcastListener) Listen() error {
    addr, _ := net.ResolveUDPAddr("udp", "239.255.0.1:7001")
    conn, _ := net.ListenMulticastUDP("udp", nil, addr)

    for {
        select {
        case <-bl.stop:
            return nil
        default:
            buf := make([]byte, 4096)
            n, _, err := conn.ReadFromUDP(buf)
            if err != nil {
                continue
            }

            var announcement ServiceAnnouncement
            json.Unmarshal(buf[:n], &announcement)

            // Process announcement
            bl.handleAnnouncement(&announcement)
```

```
        }
    }
}
```

Fallback Strategies

Strategy Hierarchy

1. **Primary**: Default port + Registry
2. **Secondary**: Broadcast discovery
3. **Tertiary**: DNS resolution
4. **Quaternary**: Configuration file
5. **Emergency**: Manual configuration override

Error Handling

Error Type	Fallback Action	Retry Logic
Port conflict	Try next available port	Immediate
Registry unavailable	Use broadcast discovery	3 retries, exponential backoff
Broadcast timeout	Try DNS resolution	After 5 seconds
DNS failure	Check configuration file	No retry
All strategies failed	Return detailed error	Manual intervention needed

Timeout Configuration

```go
type DiscoveryTimeouts struct {
    DefaultPortCheck  time.Duration // 100ms
    RegistryLookup    time.Duration // 1s
    BroadcastWait     time.Duration // 5s
    DNSResolution     time.Duration // 2s
    OverallTimeout    time.Duration // 10s
}
```

Graceful Degradation

When service discovery partially fails:

1. **Use cached endpoints** if available (with staleness check)
2. **Log warnings** but continue operation

3. **Trigger background refresh** to update stale entries
4. **Emit metrics** for monitoring and alerting

Testing Strategy

Test Coverage Requirements

All components must achieve **90%+ test coverage** across:

1. **Unit Tests**: Individual component logic
2. **Integration Tests**: Component interactions
3. **E2E Tests**: Full discovery workflows
4. **Automation Tests**: Real Docker environment
5. **Performance Tests**: Scalability and latency
6. **Chaos Tests**: Failure scenarios

Test Scenarios

Unit Tests (`internal/discovery/*_test.go`)

1. Port Allocator:
    - Default port available
    - Default port occupied → fallback
    - All ports in range occupied → ephemeral
    - Invalid port ranges
    - Concurrent port allocation
    - Port release and reallocation
2. Service Registry:
    - Service registration
    - Service deregistration
    - Duplicate registration handling
    - TTL expiration
    - Concurrent registrations
    - Health check integration
3. Discovery Client:
    - Successful discovery (all strategies)
    - Strategy fallback chain
    - Timeout handling
    - Retry logic with backoff
    - Cache hit/miss scenarios
    - Concurrent discovery requests
4. Broadcast Service:
    - Message serialization/deserialization

- UDP multicast send/receive
- Message validation
- TTL expiration
- Malformed message handling

Integration Tests (`test/integration/discovery_integration_test.go`)

1. Full Discovery Workflow:
   - Service starts → allocates port → registers → broadcasts
   - Client discovers service via registry
   - Client discovers service via broadcast
   - Multiple services with dependencies
2. Port Conflict Resolution:
   - Start two PostgreSQL instances → both get unique ports
   - Verify dependent services find correct instances
3. Configuration Broadcasting:
   - Service updates configuration → broadcast sent
   - Dependent services receive update → reconnect
4. Health Monitoring:
   - Service becomes unhealthy → registry updated
   - Clients discover alternate healthy instance

E2E Tests (`tests/e2e/testbank/discovery/`)

1. Multi-Service Environment:
   - Start full HelixCode stack (DB, Redis, API, Workers)
   - All services discover each other automatically
   - Verify end-to-end functionality
2. Docker Environment:
   - Run services in Docker containers
   - Test bridge network discovery
   - Test host network discovery
3. Failure Scenarios:
   - Kill service mid-operation → clients failover
   - Network partition → broadcast fails → registry works
   - Registry crash → broadcast takes over

Automation Tests (`test/automation/discovery_automation_test.go`)

1. Real Infrastructure Tests:
   - Spawn actual PostgreSQL and Redis containers
   - Test dynamic port allocation
   - Test service discovery across containers
   - Test configuration updates

2. Performance Tests:
    - 1000 concurrent discovery requests
    - Measure latency for each strategy
    - Test registry under load
3. Chaos Tests:
    - Random service restarts
    - Random network delays
    - Random port conflicts
    - Verify system recovers automatically

Test Implementation Example

```
// Integration test
func TestDynamicPortAllocation_Integration(t *testing.T) {
    // Start port allocator
    allocator := discovery.NewPortAllocator()

    // Start two PostgreSQL instances
    port1, err := allocator.AllocatePort("postgres-1", 5432)
    require.NoError(t, err)
    assert.Equal(t, 5432, port1) // First gets default

    port2, err := allocator.AllocatePort("postgres-2", 5432)
    require.NoError(t, err)
    assert.Equal(t, 5433, port2) // Second gets fallback

    // Start actual Docker containers
    container1 := startPostgres(t, port1)
    container2 := startPostgres(t, port2)
    defer container1.Stop()
    defer container2.Stop()

    // Register services
    registry := discovery.NewRegistry()
    registry.Register(&discovery.ServiceEndpoint{
        Name: "postgres-1",
        Port: port1,
        Host: "localhost",
    })
    registry.Register(&discovery.ServiceEndpoint{
        Name: "postgres-2",
        Port: port2,
        Host: "localhost",
    })
```

```
    // Client discovers services
    client := discovery.NewClient(registry)

    endpoint1, err := client.Discover("postgres-1")
    require.NoError(t, err)
    assert.Equal(t, port1, endpoint1.Port)

    endpoint2, err := client.Discover("postgres-2")
    require.NoError(t, err)
    assert.Equal(t, port2, endpoint2.Port)

    // Verify actual connectivity
    conn1, err := sql.Open("postgres", buildDSN(endpoint1))
    require.NoError(t, err)
    assert.NoError(t, conn1.Ping())

    conn2, err := sql.Open("postgres", buildDSN(endpoint2))
    require.NoError(t, err)
    assert.NoError(t, conn2.Ping())
}
```

Implementation Plan

Phase 1: Core Infrastructure (Week 1)

Deliverables: - [ ] Port Allocator implementation - [ ] Service Registry implementation - [ ] Basic Discovery Client - [ ] Unit tests (90% coverage)

Files: - `internal/discovery/port_allocator.go` - `internal/discovery/registry.go` - `internal/discovery/client.go` - `internal/discovery/*_test.go`

Phase 2: Broadcasting & Health (Week 2)

Deliverables: - [ ] Broadcast Service implementation - [ ] Configuration Manager - [ ] Health Monitor - [ ] Integration tests

Files: - `internal/discovery/broadcast.go` - `internal/discovery/config_manager.go` - `internal/discovery/health.go` - `test/integration/discovery_integration_test.go`

Phase 3: Docker Integration (Week 3)

Deliverables: - [ ] Update Docker Compose files - [ ] Update docker-compose.test.yml with dynamic ports - [ ] Update docker-compose.e2e.yml - [ ] Automation tests

Files: - `docker-compose.test.yml` - `tests/e2e/docker/docker-compose.e2e.yml` - `test/automation/discovery_automation_test.go`

Phase 4: Service Updates (Week 4)

Deliverables: - [ ] Update Database package to use discovery - [ ] Update Redis package to use discovery - [ ] Update Server package to use discovery - [ ] E2E tests

Files: - `internal/database/database.go` - `internal/redis/redis.go` - `internal/server/server.go` - `tests/e2e/testbank/discovery/`

Phase 5: Documentation & Polish (Week 5)

Deliverables: - [ ] Complete API documentation - [ ] User guide - [ ] Troubleshooting guide - [ ] Performance tuning guide - [ ] Migration guide for existing deployments

Files: - `docs/Architecture/DYNAMIC_PORT_BINDING_AND_SERVICE_DISCOVERY.md` (this file) - `docs/UserGuides/SERVICE_DISCOVERY_GUIDE.md` - `docs/Troubleshooting/SERVICE_DISCOVERY.md`

API Reference

Port Allocator API

```go
// NewPortAllocator creates a new port allocator
func NewPortAllocator(config PortAllocatorConfig) *PortAllocator

// AllocatePort allocates a port for a service
func (pa *PortAllocator) AllocatePort(serviceName string,
        preferredPort int) (int, error)

// AllocatePortInRange allocates a port within a specific range
func (pa *PortAllocator) AllocatePortInRange(serviceName string,
        start, end int) (int, error)

// ReleasePort releases a previously allocated port
```

```
func (pa *PortAllocator) ReleasePort(port int) error

// IsPortAvailable checks if a port is available for binding
func (pa *PortAllocator) IsPortAvailable(port int) bool

// GetPortForService returns the port allocated to a service
func (pa *PortAllocator) GetPortForService(serviceName string)
        (int, bool)
```

Service Registry API

```
// NewRegistry creates a new service registry
func NewRegistry(config RegistryConfig) *Registry

// Register registers a service endpoint
func (r *Registry) Register(endpoint *ServiceEndpoint) error

// Deregister removes a service endpoint
func (r *Registry) Deregister(serviceID string) error

// Lookup finds service endpoints by name
func (r *Registry) Lookup(serviceName string)
        ([]*ServiceEndpoint, error)

// Get retrieves a specific service endpoint by ID
func (r *Registry) Get(serviceID string) (*ServiceEndpoint,
        error)

// List returns all registered services
func (r *Registry) List() ([]*ServiceEndpoint, error)

// UpdateHealth updates the health status of a service
func (r *Registry) UpdateHealth(serviceID string, healthy bool)
        error
```

Discovery Client API

```
// NewClient creates a new discovery client
func NewClient(config ClientConfig) (*Client, error)

// Discover discovers a single service instance
func (c *Client) Discover(serviceName string,
        opts ...DiscoverOption) (*ServiceEndpoint, error)

// DiscoverAll discovers all instances of a service
```

```go
func (c *Client) DiscoverAll(serviceName string)
        ([]*ServiceEndpoint, error)

// Watch watches for service changes
func (c *Client) Watch(serviceName string) (<-chan ServiceEvent,
        error)

// Close closes the discovery client
func (c *Client) Close() error
```

Broadcast Service API

```go
// NewBroadcaster creates a new broadcast service
func NewBroadcaster(config BroadcastConfig) (*Broadcaster, error)

// Announce broadcasts a service announcement
func (b *Broadcaster) Announce(action string, endpoint
        *ServiceEndpoint) error

// Listen starts listening for announcements
func (b *Broadcaster) Listen() (<-chan *ServiceAnnouncement,
        error)

// Stop stops the broadcaster
func (b *Broadcaster) Stop() error
```

Monitoring & Observability

Metrics

The discovery system exposes the following metrics (Prometheus format):

```
# Port Allocation
discovery_ports_allocated_total{service_type="database"}
discovery_ports_released_total{service_type="database"}
discovery_port_conflicts_total{service_type="database"}

# Service Registry
discovery_services_registered_total{service_name="postgres"}
discovery_services_deregistered_total{service_name="postgres"}
discovery_service_lookups_total{service_name="postgres",result="success|fa
discovery_service_lookup_duration_seconds{service_name="postgres"}

# Discovery Client
```

```
discovery_requests_total{strategy="registry|broadcast|dns",result="success
discovery_request_duration_seconds{strategy="registry"}
discovery_cache_hits_total{service_name="postgres"}
discovery_cache_misses_total{service_name="postgres"}

# Broadcast Service
discovery_announcements_sent_total{action="register|deregister|heartbeat"}
discovery_announcements_received_total{action="register|deregister|heartbe
discovery_broadcast_errors_total{type="send|receive"}

# Health
discovery_health_checks_total{service_name="postgres",result="success|fail
discovery_unhealthy_services{service_name="postgres"}
```

Logging

Structured logging using `log/slog`:

```go
slog.Info("Service registered",
    "service_name", endpoint.Name,
    "service_id", endpoint.ID,
    "port", endpoint.Port,
    "protocol", endpoint.Protocol,
)

slog.Warn("Port conflict detected, using fallback",
    "service_name", serviceName,
    "preferred_port", preferredPort,
    "allocated_port", allocatedPort,
)

slog.Error("Discovery failed after all strategies",
    "service_name", serviceName,
    "strategies_tried", []string{"registry", "broadcast", "dns"},
    "error", err,
)
```

Security Considerations

Port Security

- **Port Range Restrictions**: Only allow allocation in configured ranges
- **Service Authentication**: Verify service identity before registration

- **Rate Limiting**: Prevent port exhaustion attacks

Broadcast Security

- **Message Signing**: Sign broadcast messages with HMAC-SHA256
- **TTL Validation**: Reject messages with expired or excessive TTL
- **Source Validation**: Verify source IP against allowed ranges

Registry Security

- **TLS Encryption**: Use TLS for registry communication
- **Access Control**: Require authentication for registry operations
- **Audit Logging**: Log all registration/deregistration events

---

Appendix A: Configuration Examples

Port Allocator Configuration

```yaml
port_allocator:
  enabled: true
  allow_ephemeral: false
  port_ranges:
    database:
      start: 5433
      end: 5442
    cache:
      start: 6380
      end: 6389
    api:
      start: 8081
      end: 8099
  reserved_ports:
    - 22    # SSH
    - 80    # HTTP
    - 443   # HTTPS
```

Service Registry Configuration

```yaml
service_registry:
  enabled: true
  listen_address: "0.0.0.0:7000"
  storage_backend: "memory"  # or "etcd", "consul"
```

```
  ttl_default: 60s
  cleanup_interval: 30s
```

Broadcast Configuration

```
broadcast:
  enabled: true
  multicast_address: "239.255.0.1:7001"
  interface: "0.0.0.0"
  ttl: 60s
  announce_interval: 30s
```

Discovery Client Configuration

```
discovery:
  enabled: true
  registry_url: "http://localhost:7000"
  strategies:
    - default
    - registry
    - broadcast
    - dns
  timeouts:
    default_port_check: 100ms
    registry_lookup: 1s
    broadcast_wait: 5s
    dns_resolution: 2s
    overall: 10s
  cache:
    enabled: true
    ttl: 5m
    max_entries: 1000
```

Appendix B: Troubleshooting

Common Issues

1. **“No ports available” error**
    - **Cause**: All ports in range are occupied
    - **Solution**: Increase port range or enable ephemeral ports
2. **“Service not found” error**
    - **Cause**: Service hasn’t registered or registry is down
    - **Solution**: Check service logs, verify registry is running

3. **Discovery timeout**
    - **Cause**: Network issues or service not responding
    - **Solution**: Check network connectivity, increase timeouts
4. **Broadcast not working**
    - **Cause**: Firewall blocking UDP multicast
    - **Solution**: Allow UDP port 7001, enable multicast on network

---

Appendix C: Performance Benchmarks

Expected Performance

Operation	Latency (p50)	Latency (p99)	Throughput
Port Allocation	< 1ms	< 5ms	10,000 ops/sec
Registry Lookup	< 10ms	< 50ms	5,000 ops/sec
Broadcast Send	< 5ms	< 20ms	1,000 msgs/sec
Discovery (cached)	< 1ms	< 5ms	50,000 ops/sec
Discovery (uncached)	< 50ms	< 200ms	1,000 ops/sec

Document Version: 1.0
Last Updated: 2025-11-07
Authors: HelixCode Core Team
Status: Design Phase - Ready for Implementation